

LAMBDA EXPRESSIONS AND INTRODUCTION TO RECURSION Solutions

COMPUTER SCIENCE MENTORS 61A

February 10, 2026 – February 13, 2026

1 Lambda Expressions

1. Draw the environment diagram that results from running the code.

```
def a(y):  
    d = 1  
    b = lambda x: y(x)  
    e = lambda x: x(3)  
    return e(b)  
d = 5  
a(lambda x: 4 - x + d)
```

<https://goo.gl/9vxEwv>

2. What would Python display?

(a) `(lambda x: x(x))(lambda y: 4)`

4

(b) `(lambda x, y: y(x))(mul, lambda a: a(3, 5))`

15

3. Make a lambda function, `make_interval()`, that takes in the upper and lower bound of an interval, and returns a function that takes in a value `x` and checks whether `x` is in the interval `[lower, upper]`, inclusive.

```
>>> make_interval = _____  
>>> in_interval = make_interval(-1, 2)  
>>> in_interval(0)  
True  
>>> in_interval(61)  
False
```

```
>>> make_interval = lambda lower, upper: lambda x: x <= upper and x >=  
    lower  
>>> in_interval = make_interval(-1, 2)  
>>> in_interval(0)  
True  
>>> in_interval(61)  
False
```

2 Introduction to Recursion

There are three steps to writing a recursive function:

1. Create base case(s)
2. Reduce your problem to a smaller subproblem and call your function recursively to solve the smaller subproblem(s)
3. Use the subproblems' solutions as pieces to construct a larger problem's solution (This can happen in many layers!)

Real World Analogy for Recursion

Imagine that you're in line for boba, but the line is really long, so you want to know what position you're in.



You decide to ask the person in front of you how many people are in front of them. That way, you can take their response and add 1 to it to find your place. Now, the person in front of you is faced with the same problem that you were trying to solve, with one less person in front of them than you. They decide to take the same approach that you did by asking the person in front of them. This continues until the very first person in line is asked. At this point, the person at the front knows that there are 0 people in front of them, so they can tell the person behind them that there are 0 people in front. Now, the second person can figure out that there is 1 person in front of them, and can relay that back to the person behind them, and so on, until the answer reaches you.

Looking at this example, we see that we have broken down the problem of "how many people are there in front of me?" to $1 +$ "how many people are there in front of the person in front of me"? This problem will terminate with the person at the front of the line (with 0 people in front of them). Putting this into more formal terms, we are breaking down the problem into a **recurrence relationship**, and the termination case (when the question gets to the very first person in line) is called the **base case**.

As a program goes through recursion, it doesn't formally solve the problem until it reaches the base case, in which case it works its way up from the base case to the original input to construct your final answer. Just like your question as you stand in line, the program:

1. goes out (down the call stack), person by person (call by call) (**recursive case**)
2. until the person at the very front is reached (**base case**)
3. when the answers get added onto and eventually passed back to you, solving the problem. (**constructing the final solution with solutions to subparts**)
4. Translate the example of standing in a line for boba into Python! We can represent our line as an integer of 0s and 1s, where the 0s represent an empty space and the 1s represent a person standing in line. You are about to stand at the end of the line, but first, you need to count how many people would be in front of you! Complete the definition for `boba_line`, which takes in an integer n and returns the number of 1s in it. Implement the function recursively, with a base case and a recursive case.

Hint: To get the last digit of an integer n , you can do $n \% 10$ To remove the last digit from an integer, you can do $n // 10$

```
def boba_line(n):
    """Takes in an integer of 0s and 1s and returns the number of 1s.

    >>> boba_line(0)
    0
    >>> boba_line(1)
    1
    >>> boba_line(10101)
    3
    >>> boba_line(10001)
    2
    """
    if _____ :                # base case

        return _____

    else:

        return _____ + _____ # recursive case

    if n < 10:
        return n
    else:
        return (n % 10) + boba_line(n // 10)
```

5. The *Gibonacci sequence* is a recursively defined sequence of integers; we denote the n th Gibonacci number g_n . The first three terms of the sequence are $g_0 = 0, g_1 = 1, g_2 = 2$. For $n \geq 3$, g_n is defined as the sum of the previous three terms in the sequence.

Complete the function `gib`, which takes in an integer `n` and returns the n th Gibonacci number, g_n . Also, identify the three parts of recursive function design as they are used in your solution.

```
def gib(n):  
    """  
    >>> gib(0)  
    0  
    >>> gib(1)  
    1  
    >>> gib(2)  
    2  
    >>> gib(3) # gib(2) + gib(1) + gib(0) = 3  
    3  
    >>> gib(4) # gib(3) + gib(2) + gib(1) = 6  
    6  
    """  
    if _____:  
        return _____  
  
    return _____
```

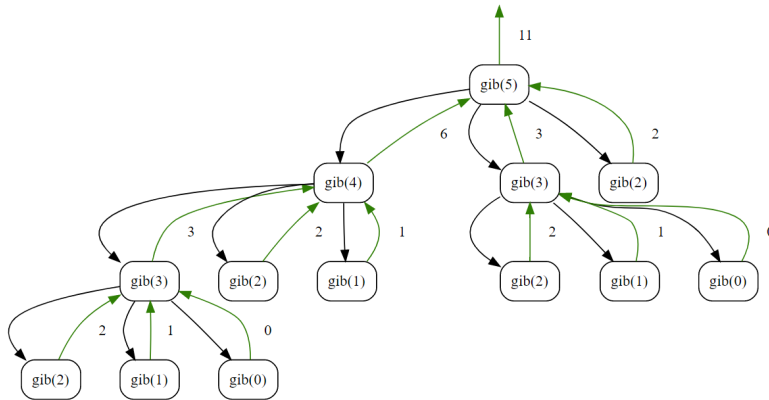
```
def gib(n):  
    if n <= 2:  
        return n  
    return gib(n - 1) + gib(n - 2) + gib(n - 3)
```

- Base case: `if n <= 2: return n`
- Recursive calls: `gib(n - 1), gib(n - 2), gib(n - 3)`
- Solving the larger problem: adding the results of the three recursive calls.

6. Including the original call, how many calls are made to `gib` when you evaluate `gib(5)`?

13.

There are a few ways you could determine this. First, you could draw out a call graph that shows all of the calls that are made. <https://tinyurl.com/gibsol>



You could also build things from the ground up using recursive intuition.

- `gib(0)` takes one call because it's a base case.
- `gib(1)` takes one call because it's a base case.
- `gib(2)` takes one call because it's a base case.
- `gib(3)` takes four calls: one for the original call, one for `gib(0)`, one for `gib(1)`, and one for `gib(2)`.
- `gib(4)` takes seven calls: one for the original call, four for `gib(3)`, one for `gib(2)`, and one for `gib(1)`.
- `gib(5)` takes 13 calls: one for the original call, seven for `gib(4)`, four for `gib(3)`, and one for `gib(2)`.

This second method is much faster, especially for higher values of `n`.

7. Complete the definition for `num_digits`, which takes in a number n and returns the number of digits it has. Implement the function recursively.

```
def num_digits(n):  
    """Takes in an positive integer and returns the number of  
    digits.  
  
    >>> num_digits(0)  
    1  
    >>> num_digits(1)  
    1  
    >>> num_digits(7)  
    1  
    >>> num_digits(1093)  
    4  
    """  
  
    if n < 10:  
        return 1  
    else:  
        return 1 + num_digits(n // 10)
```

8. Implement a recursive version of fizzbuzz.

```
def fizzbuzz(n):  
    """Prints the numbers from 1 to n. If the number is divisible by 3, it  
    instead prints 'fizz'. If the number is divisible by 5, it instead  
    prints  
    'buzz'. If the number is divisible by both, it prints 'fizzbuzz'. You  
    must do this recursively!  
  
    >>> fizzbuzz(15)  
    1  
    2  
    fizz  
    4  
    buzz  
    fizz  
    7  
    8  
    fizz  
    buzz  
    11  
    fizz  
    13  
    14  
    fizzbuzz  
    """  
  
    if n == 1:  
        print(n)  
    else:  
        fizzbuzz(n - 1)  
        if n % 3 == 0 and n % 5 == 0:  
            print('fizzbuzz')  
        elif n % 3 == 0:  
            print('fizz')  
        elif n % 5 == 0:  
            print('buzz')  
        else:  
            print(n)
```